Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение высшего образования «Омский государственный технический университет»
Кафедра «Автоматизированные системы обработки информации и управления»

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

по дисциплине «Кроссплатформенная разработка»

Сафронов Александр Александрович ПИН-252т

Пояснительная записка
Шифр работы От-43-ПИН-252т-2 ЛР
Специальность 09.03.03 Программная инженерия

Преподаватель М.А. Мищенко
Студент А.А. Сафронов

Омск 2026

РЕФЕРАТ

Объектом работы является каркас кроссплатформенного приложения «Medicine Publications» на Flutter для просмотра каталога научных публикаций PubMed. Источником данных служит локальный набор из 20 статей.

Цель работы - реализовать список публикаций с поиском, экран статьи со связанными публикациями и навигацию между экранами на единой кодовой базе.

Применены разделение на слои domain, data и presentation, управление состоянием через Cubit и Bloc, библиотека go_router, цветовые схемы Material 3 и локализация на основе ARB-файлов.

Степень внедрения - для учебных целей. Каркас подготовлен для последующего подключения источника данных NCBI; сетевое взаимодействие в данной работе не реализовано.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4
1 ПОСТАНОВКА ЗАДАЧИ 5
2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ 6
3 РЕАЛИЗАЦИЯ 8
3.1 Модель публикации и репозиторий 8
3.2 Моковые данные и связанные статьи 8
3.3 Список публикаций и поиск 9
3.4 Состояние экрана статьи 11
3.5 Навигация и обработка адресов 11
3.6 Тема и общие компоненты 13
3.7 Локализация интерфейса 14
4 ПРОВЕРКА РАБОТЫ 15
4.1 Статический анализ проекта 15
4.2 Сценарии проверки интерфейса 16
4.3 Внешний вид приложения 16
ЗАКЛЮЧЕНИЕ 20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 21

ВВЕДЕНИЕ

Flutter позволяет создавать интерфейс приложения на языке Dart для нескольких платформ из общей кодовой базы [1]. В лабораторной работе № 2 реализован каркас каталога медицинских публикаций «Medicine Publications»: список с поиском, подробные сведения о статье и переходы к связанным материалам.

Постановка задачи и требования к каркасу приняты из отчёта лабораторной работы № 1. Вторая работа сохраняет назначение приложения, но использует средства Flutter: виджеты, Cubit и Bloc, go_router, ThemeData и генерацию локализации. Способы реализации требований и отличия рассмотрены в разделе 1.

Цель работы - построить каркас приложения с разделением данных, предметной модели и интерфейса. Для достижения цели требуется подготовить моки, реализовать два основных экрана, поиск, стек переходов, две темы и два языка интерфейса, а также проверить доступные результаты.

Данные каталога доступны без сети через интерфейс репозитория. NCBI Entrez Programming Utilities рассматривается как источник для следующего этапа [5]. В текущем проекте HTTP-запросы, загрузка статей из PubMed и получение связей через API отсутствуют.

1 ПОСТАНОВКА ЗАДАЧИ

Разрабатываемое приложение содержит экран списка и экран статьи. Список должен показывать карточки публикаций и поддерживать поиск. Экран статьи должен выводить тип публикации, название, дату, PMID, описание, авторов и связанные статьи. Открытие связанного материала добавляет экран в стек, возврат закрывает один экран.

Требования к каркасу и их выполнение во Flutter приведены в таблице 1. Сохранено назначение требований ЛР 1; технологические замены и ограничения указаны явно.

Таблица 1 - Требования к каркасу приложения

Требование	Flutter-реализация и соответствие
Навигация Navigation 3, маршрут-объект	go_router, путь /article/:id, AppRoutes.detail(int). Маршрут является строкой, а не объектом. Числовой PMID проверяется через int.tryParse.
Возврат по одному экрану	ArticleCard вызывает context.push. Вложенный маршрут статьи сохраняет список под ней. Специальные спецификации анимаций не заданы.
Экран рисуется из состояния	BlocBuilder и switch по sealed-состояниям. Для списка действия представлены методами Cubit, для статьи - событием Bloc.
Связанные публикации	PMID заданы в relatedIds. ArticleDetailBloc получает записи через репозиторий. Правило совпадения автора или категории из ЛР 1 не применяется.
Строковые ресурсы, две локали	ARB-файлы ru и en, генерация AppLocalizations. LocaleCubit переключает язык кнопкой.
Тема из токенов оформления	ThemeData и ColorScheme.fromSeed; AppColors хранит опорный цвет и цвета типов. Светлая и тёмная схемы.
Отчёт компилятора включён	Прямого соответствия нет. Применён flutter analyze для статической проверки кода; сведения о пропуске перерисовок он не формирует.

Код разделён на domain, data и presentation. Общая кодовая база предназначена для Android, iOS, macOS и web.

2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ

В проекте применена организация по функциональным возможностям. Каталог lib/features/article содержит три слоя. Слой domain определяет ArticleModel, ArticleType и IArticleRepository, не зависит от Flutter и не содержит виджетов. Слой data реализует интерфейс репозитория на локальных данных. Слой presentation содержит управление состоянием, экраны, компоненты и форматирование модели для интерфейса.

Общие механизмы вынесены в lib/common: navigation, theme, localization и widgets. ARB-файлы находятся отдельно в lib/l10n. Точка входа lib/main.dart вызывает runApp(const App()). Корневой виджет описан в lib/app.dart.

Зависимости передаются через провайдеры библиотеки flutter_bloc [2]. IArticleRepository регистрируется в RepositoryProvider, а ArticleListCubit, ThemeCubit и LocaleCubit создаются в MultiBlocProvider. Фрагмент регистрации приведён в листинге 1.

Файл: lib/app.dart

```
return MultiRepositoryProvider(
      providers: [
        RepositoryProvider<IArticleRepository>(
          create: (_) => const ArticleRepository(),
        ),
      ],
      child: MultiBlocProvider(
        providers: [
          BlocProvider(
            create: (context) =>
                ArticleListCubit(context.read<IArticleRepository>())..load(),
          ),
          BlocProvider(create: (_) => ThemeCubit()),
          BlocProvider(create: (_) => LocaleCubit()),
        ],
```

Листинг 1 - Внедрение репозитория и создание Cubit

Cubit списка расположен выше MaterialApp.router и сохраняется при переходах между статьями. Он хранит строку запроса и результат поиска. Позиция прокрутки не является полем Cubit: она относится к состоянию ListView и сохранённого маршрута списка. Её сохранение после push и воз-

врата необходимо проверить отдельно. После перезапуска приложения данные Cubit не восстанавливаются из постоянного хранилища.

ArticleDetailBloc создаётся локальным BlocProvider в ArticleDetailScreen. У каждой страницы статьи собственный экземпляр, которому передаётся PMID через ArticleDetailOpened. При удалении страницы провайдер освобождает созданный Bloc.

Поток обработки имеет вид: действие пользователя - Cubit или событие Bloc - IArticleRepository - ArticleRepository - новое состояние - BlocBuilder. Навигация вызывается виджетами. Cubit и Bloc не принимают BuildContext и не управляют маршрутами.

3 РЕАЛИЗАЦИЯ

3.1 Модель публикации и репозиторий

Модель ArticleModel использует идентификатор PubMed типа int и список авторов. Дополнительно заданы название, описание, тип, дата публикации и PMID связанных статей. Фрагмент модели приведён в листинге 2.

Файл: lib/features/article/domain/article_model.dart

```
final int pmid;
  final List<String> authors;
  final String title;
  final String description;
  final ArticleType type;
  final DateTime publishedAt;
  final List<int> relatedIds;
}
```

Листинг 2 - Поля модели ArticleModel

ArticleType задаёт четыре варианта: research, review, randTrial и validStudy. Подписи, значки и цвета для отображения определены расширением ArticleTypeFormat в presentation/utils/article_type_format.dart. Предметная модель не зависит от классов оформления.

Контракт доступа к данным приведён в листинге 3. Методы возвращают Future, поэтому реализацию на моках можно заменить асинхронным источником с сохранением контракта. Для такой замены потребуется новая реализация data и её регистрация в app.dart; обработку сетевых ошибок также потребуется добавить.

Файл: lib/features/article/domain/i_article_repository.dart

```
abstract interface class IArticleRepository {
  Future<List<ArticleModel>> getArticles();
  Future<ArticleModel?> getArticle(int pmid);
  Future<List<ArticleModel>> searchArticles(String query);
}
```

Листинг 3 - Интерфейс IArticleRepository

3.2 Моковые данные и связанные статьи

В lib/features/article/data/mock_articles.dart подсчитано 20 конструкторов ArticleModel и 20 значений pmid. Набор включает публикации о COVID-19, лекарственной терапии ожирения, редактировании генома и применении ис-

кусственного интеллекта в медицине. Данные встроены в приложение и не запрашиваются из сети.

ArticleRepository.getArticles возвращает mockArticles, а getArticle выполняет поиск по PMID и возвращает null при отсутствии записи. Связи хранятся непосредственно в relatedIds. Например, у статьи 37952131 указаны PMID 33567185, 35658024 и 27633186.

ArticleDetailBloc параллельно получает связанные записи через Future.wait, исключает отсутствующие записи с помощью whereType<ArticleModel> и передаёт результат в ArticleDetailLoaded. Значимый фрагмент приведён в листинге 4.

Файл: lib/features/article/presentation/bloc/detail/article_detail_bloc.dart

```
final article = await _repository.getArticle(event.pmid);
    if (article == null) {
      emit(ArticleDetailNotFound(event.pmid));
      return;
    }
    final found = await Fu-
ture.wait(article.relatedIds.map(_repository.getArticle));
    final related = found.whereType<ArticleModel>().toList();
    emit(ArticleDetailLoaded(article, related: related));
  }
}
```

Листинг 4 - Загрузка статьи и связанных публикаций

Связь не вычисляется по общему автору или типу публикации. При отсутствии связанных записей интерфейс выводит relatedEmpty. Обложки представлены значками Material Icons, выбранными по типу публикации; сетевые изображения и отдельный источник картинок в проекте отсутствуют.

3.3 Список публикаций и поиск

ArticleListScreen получает ArticleListCubit из корня и передаёт в SearchField текущий query и метод search. Контроллер поля создаётся в initState и освобождается в dispose. Очистка поля вызывает обработчик с пустой строкой.

Поиск реализован в ArticleRepository.searchArticles. Запрос очищается от крайних пробелов и переводится в нижний регистр. Совпадение ищется по

подстроке в названии, описании и каждом имени автора. Пустой запрос возвращает полный каталог. Предикат приведён в листинге 5.

Файл: lib/features/article/data/article_repository.dart

```
static bool _matches(ArticleModel article, String needle) =>
      article.title.toLowerCase().contains(needle) ||
      article.description.toLowerCase().contains(needle) ||
      article.authors.any((author) => author.toLowerCase().contains(needle));
}
```

Листинг 5 - Поля поиска в ArticleRepository

Для списка определены ArticleListLoading, ArticleListLoaded и ArticleListNotFound. В базовом ArticleListState хранится query. ArticleListLoaded дополнительно содержит articles. Экран выбирает представление через switch: индикатор загрузки, NothingFound или список карточек со счётчиком.

ArticleListCubit.load вызывает search с пустой строкой. Новые поисковые запросы не переводят экран обратно в Loading. После завершения Future проверяется актуальность строки и состояние закрытия Cubit. Реализация приведена в листинге 6.

Файл: lib/features/article/presentation/bloc/list/article_list_cubit.dart

```
String _latestQuery = '';
  Future<void> load() => search('');
  Future<void> search(String query) async {
    _latestQuery = query;
    final isEmptyQuery = query.trim().isEmpty;
    final articles = isEmptyQuery
        ? await _repository.getArticles()
        : await _repository.searchArticles(query);
    if (isClosed || query != _latestQuery) return;
    emit(
      articles.isEmpty && !isEmptyQuery
          ? ArticleListNotFound(query: query)
          : ArticleListLoaded(query: query, articles: articles),
    );
  }
}
```

Листинг 6 - Поиск и защита от устаревшего ответа

Проверка query != _latestQuery отбрасывает ответ для отличающейся предыдущей строки. Это не отменяет Future и не является уникальной нумерацией запросов: при последовательности A, B, A ранний ответ A не различается с последним A. Для текущего репозитория на постоянных моках это не меняет результат поиска.

Карточки строятся в ListView.separated. В ArticleCard выводятся TypeChip, значок обложки, название не более трёх строк и строка автора с годом. Ограничение ширины списка равно 840 логическим пикселям, боковые поля вычисляются в LayoutBuilder и составляют не менее 16.

3.4 Состояние экрана статьи

ArticleDetailBloc регистрирует обработчик ArticleDetailOpened. Начальное состояние - ArticleDetailLoading. Если репозиторий возвращает null, создаётся ArticleDetailNotFound с запрошенным PMID; иначе - ArticleDetailLoaded со статьёй и связанными записями.

ArticleDetailState объявлен sealed-классом. Отдельных состояний для сетевой ошибки нет, поскольку текущая реализация работает на локальном наборе. Разбор состояния экраном приведён в листинге 7.

Файл: lib/features/article/presentation/view/article_detail_screen.dart

```
builder: (context, state) => switch (state) {
            ArticleDetailLoading() => const Center(
              child: CircularProgressIndicator(),
            ),
            ArticleDetailNotFound(:final pmid) => Center(
              child: Text(
                AppLocalizations.of(context).detailNotFound(pmid.toString()),
              ),
            ),
            ArticleDetailLoaded(:final article, :final related) =>
              _DetailContent(article: article, related: related),
          },
```

Листинг 7 - Отображение состояний ArticleDetailScreen

В загруженном представлении используются SingleChildScrollView и ConstrainedBox с maxWidth 760. Выводятся тип, SelectableText с названием, ArticleFacts с датой и PMID, описание, список авторов и связанные карточки. Дата форматируется локализацией, авторы соединяются через запятую. Для даты и PMID применён Wrap, допускающий перенос элементов в узком окне.

3.5 Навигация и обработка адресов

В AppRoutes определены корневой путь / и метод detail(int pmid). AppRouter использует GoRouter [3]. Маршрут article/:id вложен в маршрут

списка; в адресе передаётся только идентификатор, модель статьи извлекается из репозитория. Определение маршрутов приведено в листинге 8.

Файл: lib/common/navigation/app_router.dart

```
abstract final class AppRoutes {
  static const String list = '/';
  static String detail(int pmid) => '/article/$pmid';
}

abstract final class AppRouter {
  static final GoRouter router = _create();

  static GoRouter _create() {
    GoRouter.optionURLReflectsImperativeAPIs = true;
    return GoRouter(
      initialLocation: AppRoutes.list,
      routes: [
        GoRoute(
          path: AppRoutes.list,
          builder: (context, state) => const ArticleListScreen(),
          routes: [
            GoRoute(
              path: 'article/:id',
              builder: (context, state) {
                final pmid = int.tryParse(state.pathParameters['id']!);
                return pmid == null
                    ? const _NotFoundScreen()
                    : ArticleDetailScreen(pmid: pmid);
              },
            ),
          ],
        ),
      ],
      errorBuilder: (context, state) => const _NotFoundScreen(),
    );
  }
}
```

Листинг 8 - Маршруты приложения

ArticleCard вызывает context.push(AppRoutes.detail(article.pmid)). Один и тот же компонент используется в списке и среди связанных статей, поэтому последовательность список - статья A - статья B создаёт стек экранов. Кнопка возврата в AppBar должна последовательно закрывать B и A. Прямую ссылку и поведение системного возврата необходимо проверить на каждой платформе.

GoRouter.optionURLReflectsImperativeAPIs включён до создания маршрутизатора, чтобы императивные переходы отражались в URL. Строка article/abc не преобразуется в число и открывает _NotFoundScreen. Для числового PMID, отсутствующего в каталоге, отображается

ArticleDetailNotFound. Неизвестный путь обрабатывается errorBuilder и показывает отдельное сообщение «Такой страницы нет».

Проверка аргумента выполняется во время исполнения. Типобезопасные классы маршрутов и генерация маршрутов в проекте не применены. Переходы используют стандартные страницы go_router; отдельные анимации прямого, обратного и предиктивного перехода не определены.

3.6 Тема и общие компоненты

AppTheme формирует светлую и тёмную ThemeData из одного AppColors.seed через ColorScheme.fromSeed. В MaterialApp.router подключены theme, darkTheme и themeMode. Код темы приведён в листинге 9.

Файл: lib/common/theme/app_theme.dart

```
abstract final class AppTheme {
  static ThemeData get light => _build(Brightness.light);
  static ThemeData get dark => _build(Brightness.dark);
  static ThemeData _build(Brightness brightness) => ThemeData(
    colorScheme: ColorScheme.fromSeed(
      seedColor: AppColors.seed,
      brightness: brightness,
    ),
    fontFamily: 'Nunito',
  );
}
```

Листинг 9 - Формирование темы

Цветовые роли Material 3 отделяют назначение цвета от конкретного значения [4]. Виджеты используют Theme.of(context).colorScheme, а цвета типов публикаций и цвет значка задаются в AppColors. Цвета типов одинаковы для обеих тем. ThemeCubit начинает со светлой темы и переключает ThemeMode.light и ThemeMode.dark.

В pubspec.yaml зарегистрирован Nunito: обычное начертание, Medium с весом 500 и Bold с весом 700. Файлы находятся в assets/fonts. Шрифт передаётся через fontFamily в ThemeData и доступен без загрузки из сети.

AppScaffold объединяет Scaffold, AppBar и кнопки переключения темы и языка. Переиспользуемые ArticleCover, TypeChip, CardSurface, Section и ArticleFacts вынесены в отдельные виджеты. Это позволяет использовать одинаковое оформление карточек в списке и на странице статьи.

3.7 Локализация интерфейса

Подписи интерфейса размещены в lib/l10n/app_ru.arb и app_en.arb. Параметр flutter.generate включён в pubspec.yaml; l10n.yaml задаёт русский шаблон app_ru.arb, выходной файл app_localizations.dart и nullable-getter: false. Генератор создаёт класс AppLocalizations.

LocaleCubit начинает с Locale('ru') и переключает ru и en. В корне подключены locale, localizationsDelegates и supportedLocales. В отличие от ЛР 1 язык задаётся состоянием приложения и кнопкой, а не только системной локалью. Фрагмент LocaleCubit приведён в листинге 10.

Файл: lib/common/localization/locale_cubit.dart

```
class LocaleCubit extends Cubit<Locale> {
  LocaleCubit({Locale initial = russian}) : super(initial);

  static const Locale russian = Locale('ru');
  static const Locale english = Locale('en');

  void toggle() {
    emit(state.languageCode == russian.languageCode ? english : russian);
  }
}
```

Листинг 10 - Переключение языка

Шаблоны listFound используют ICU plural: русские формы one, few, many и other, английские one и other. Для articlePublished задан параметр DateTime с форматом yMMMM. Строка articleByline принимает authors и year. Переводится интерфейс, включая названия типов; заголовки, описания и имена авторов остаются на языке данных.

4 ПРОВЕРКА РАБОТЫ

4.1 Статический анализ проекта

Проверка относится к Flutter-проекту «Medicine Publications» в ветке feature/lab-2_frame репозитория CancelTell/medicine-publications-flutter. Для оценки качества исходного кода применён статический анализатор Flutter.

Актуальный стабильный SDK Flutter 3.47.9 выбран в качестве рекомендуемой версии для воспроизведения проекта [6]. Метаданные локальной подготовки проекта фиксируют Flutter 3.47.4 и Dart 3.13.3. В pubspec.yaml задано ограничение Dart ^3.13.3, а pubspec.lock фиксирует flutter_bloc 9.1.1 и go_router 18.0.2.

Согласно журналу выполнения команды flutter analyze, анализ проекта завершился без диагностических замечаний: «No issues found! (ran in 0.4s)». Продолжительность данного прогона составила 0,4 с. Статический анализ проверяет исходный код и правила анализа, но не заменяет проверку пользовательских сценариев.

Результаты проверки исходного кода и предоставленных изображений интерфейса приведены в таблице 2.

Таблица 2 - Результаты проверки

Объект проверки	Результат
Моковый каталог	В mock_articles.dart содержится 20 записей ArticleModel с PMID. На снимках списка отображается счётчик «20 статей».
Статический анализ	flutter analyze: No issues found! (ran in 0.4s).
Экран статьи	На снимках представлены название, дата, PMID, описание, авторы и связанные публикации.
Узкое окно	На предоставленных изображениях карточки и текст перестраиваются по доступной ширине.
Тема и локализация	Светлая тема с русскими подписями и тёмная тема с английскими подписями представлены на изображениях.
iOS-симулятор	Представлены экран списка и экран статьи в рамке iPhone.

Статический анализ Flutter отличается от отчёта компилятора Compose, предусмотренного в ЛР 1. Команда flutter analyze выявляет диагностические замечания в коде, но не формирует отметки restartable и skippable и не измеряет пропуск построения виджетов.

4.2 Сценарии проверки интерфейса

Сценарии проверки основных функций и ожидаемые результаты приведены в таблице 3. Они охватывают поиск, открытие статей, возврат, обработку адресов и переключение оформления.

Таблица 3 - Сценарии ручной проверки

Действие	Ожидаемый результат
Открыть список	Показаны 20 карточек и счётчик; список прокручивается.
Ввести часть названия, описания или автора	Показаны совпадения без учёта регистра. После очистки возвращаются все 20 статей.
Ввести отсутствующую строку	Показано сообщение NothingFound с запросом и подсказкой.
Открыть статью, затем связанную и дважды вернуться	Возврат к первой статье, затем к списку. Проверить сохранение запроса и прокрутки.
Открыть /article/1, /article/abc и /unknown	Для отсутствующего PMID - сообщение о статье; для неверного формата и неизвестного пути - сообщение о странице.
Переключить тему и язык	Оформление меняется; подписи переключаются ru/en, данные статей не переводятся.
Сузить окно и открыть прямую ссылку	Нет переполнения; адрес статьи отражён в URL, под статьёй доступен список.

4.3 Внешний вид приложения

Экран списка в широком окне приведён на рисунке 1. Видны поле поиска, счётчик из 20 статей, цветовые метки типов и карточки с названиями и авторами публикаций.

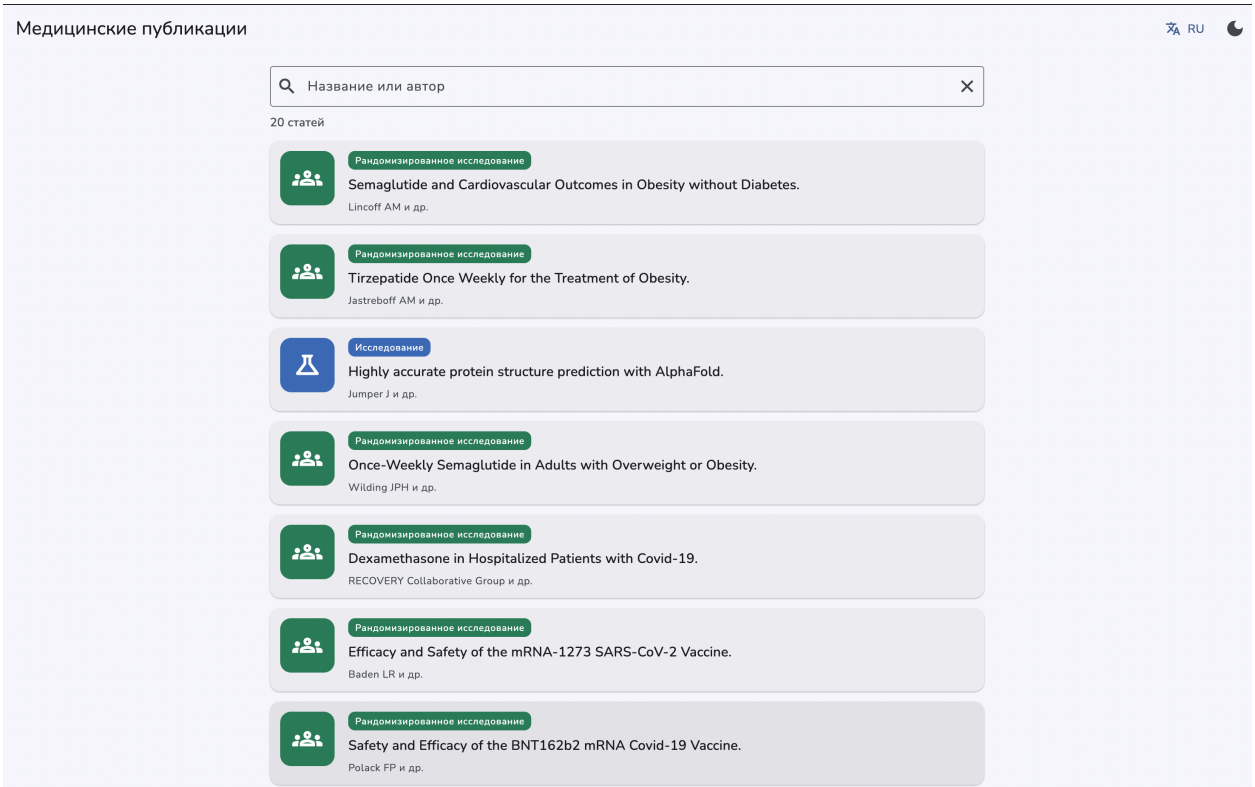


Рисунок 1 - Список публикаций в широком окне

Экран статьи приведён на рисунке 2. Отображаются тип публикации, дата, PMID, описание, список авторов и связанные статьи. Кнопка возврата расположена в верхней панели.

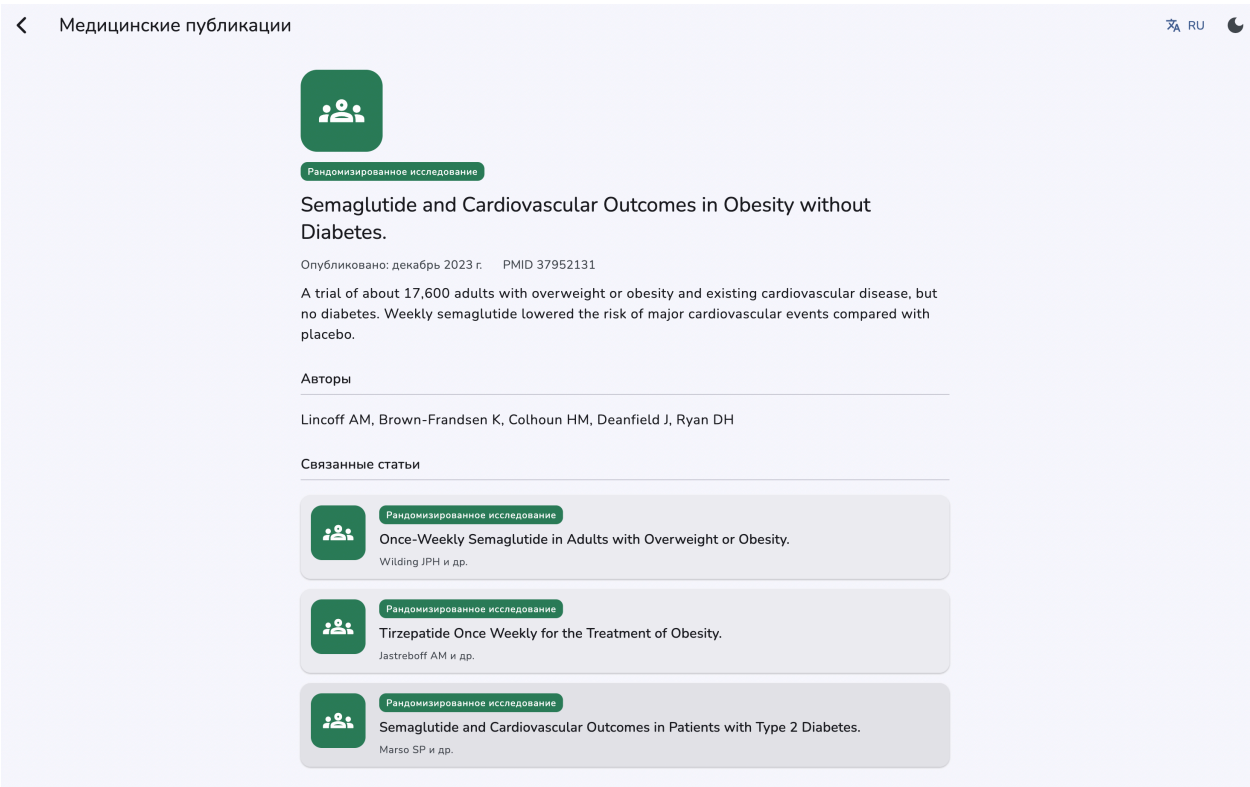


Рисунок 2 - Экран статьи и связанные публикации

Интерфейс в узком окне приведён на рисунке 3. Слева показан список в светлой теме на русском языке; справа - прокрученный экран статьи в тёмной теме на английском языке. Названия переносятся на несколько строк, а связанные публикации используют тот же компонент карточки.

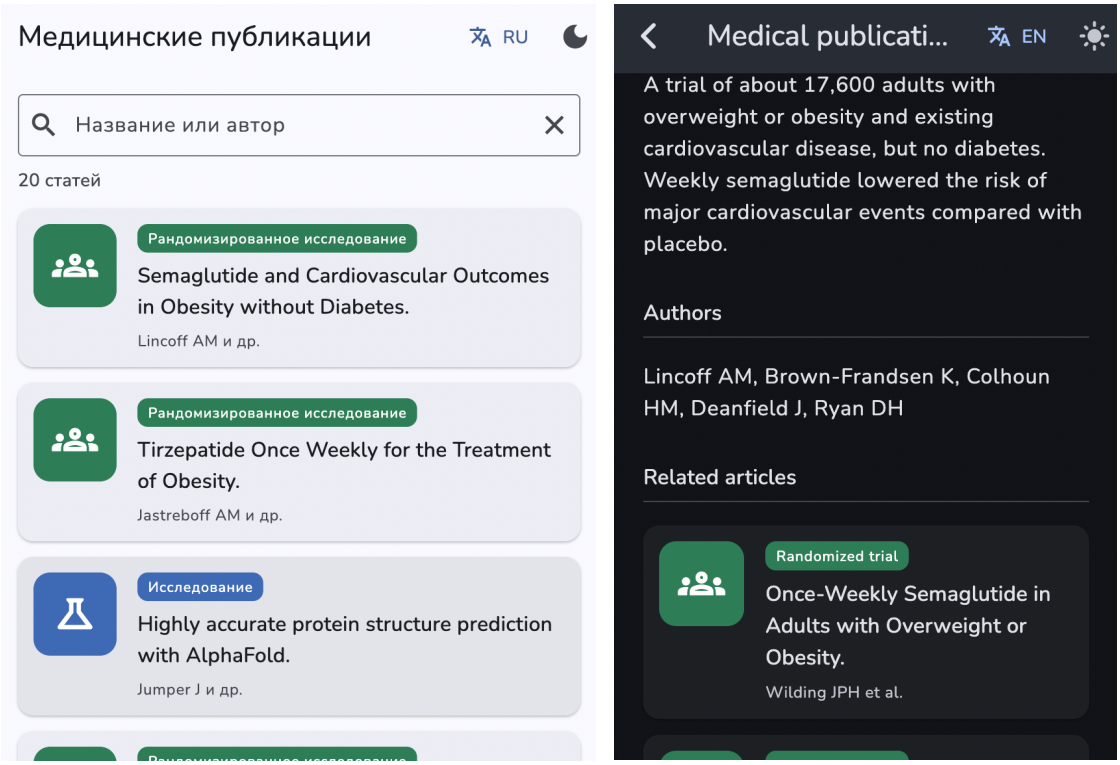


Рисунок 3 - Интерфейс приложения в узком окне

Работа приложения в iOS-симуляторе приведена на рисунке 4. Слева представлен список со светлой темой и русскими подписями, справа - статья с тёмной темой и английскими подписями. Переведены элементы интерфейса, в том числе тип, дата и заголовки разделов; данные публикации остаются на языке источника.

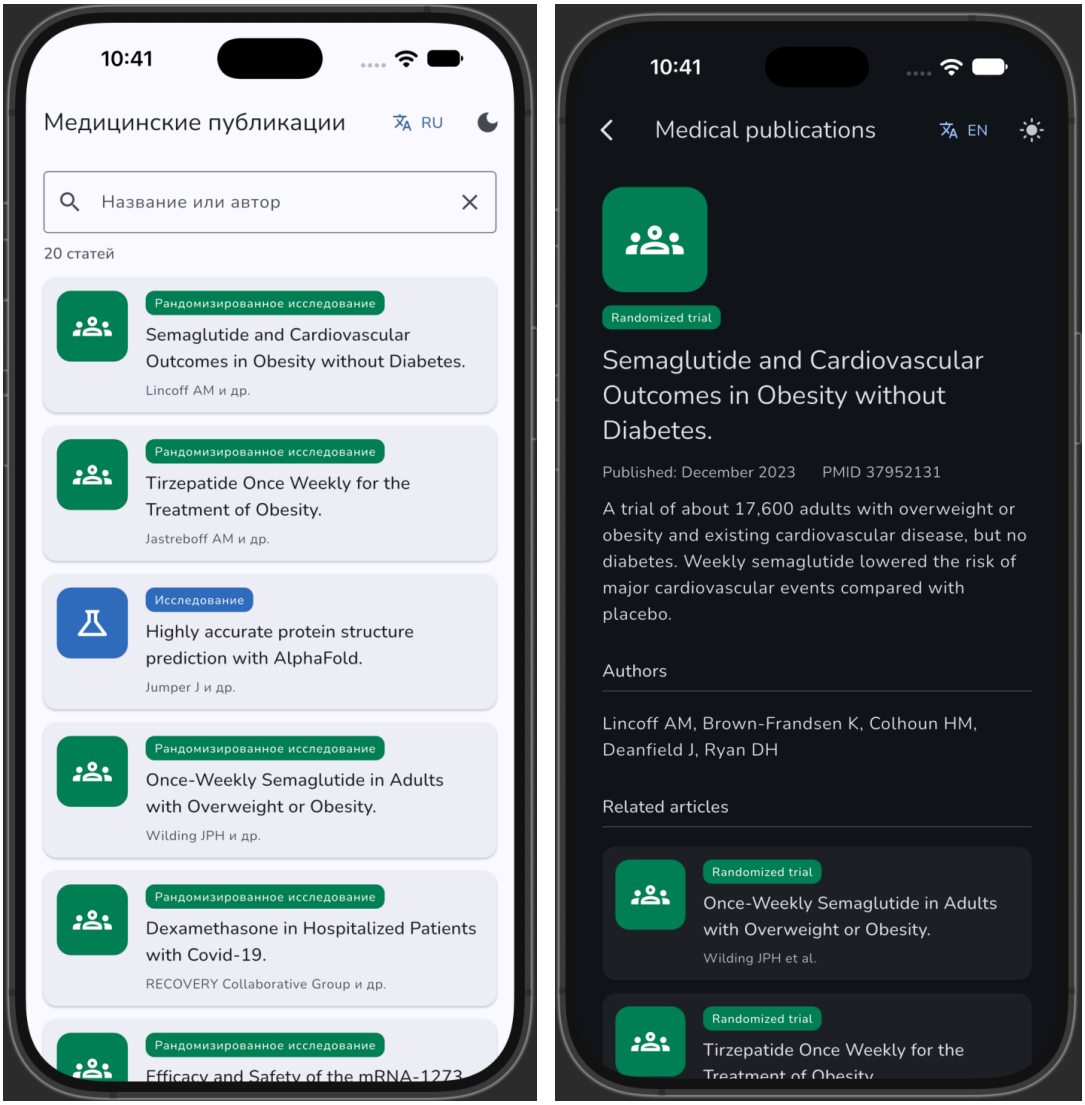


Рисунок 4 - Список и статья в iOS-симуляторе

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы реализован каркас приложения «Medicine Publications» на Flutter. Подготовлены список с поиском по названию, описанию и авторам, экран статьи со связанными публикациями, навигация через go_router, светлая и тёмная темы, русский и английский интерфейсы.

Код разделён на domain, data и presentation. Доступ к 20 моковым статьям скрыт за IArticleRepository. Состоянием списка управляет ArticleListCubit, состоянием отдельной страницы - ArticleDetailBloc. Связи задаются списком PMID.

Требования к каркасу перенесены на средства Flutter: управление состоянием выполнено через Cubit и Bloc, навигация - через go_router, тема - через ThemeData и ColorScheme, локализация - через ARB и AppLocalizations. Связанные статьи заданы PMID в relatedIds. Технологические отличия от исходной постановки указаны в таблице требований.

По журналу flutter analyze диагностических замечаний не выявлено. Представленные изображения демонстрируют список, сведения о статье, размещение элементов в узком окне, смену темы и языка интерфейса, а также работу в iOS-симуляторе. Подготовленный каркас служит основой для следующего этапа с подключением NCBI Entrez Programming Utilities.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Flutter documentation [Электронный ресурс]. - URL: https://docs.flutter.dev/

2. flutter_bloc: документация пакета [Электронный ресурс]. - URL: https://pub.dev/packages/flutter_bloc

3. go_router: документация пакета [Электронный ресурс]. - URL: https://pub.dev/packages/go_router

4. Material Design 3: Color system [Электронный ресурс]. - URL: https://m3.material.io/styles/color/system/overview

5. Entrez Programming Utilities Help [Электронный ресурс]. - URL: https://www.ncbi.nlm.nih.gov/books/NBK25501/

6. Flutter stable changelog [Электронный ресурс]. - URL: https://github.com/flutter/flutter/blob/stable/CHANGELOG.md